



智能计算系统

第六章 面向深度学习的 处理器原理

北京邮电大学 人工智能学院

何召锋、徐振博、项刘宇

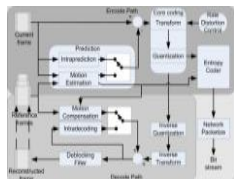
提纲

- ▶ 通用处理器
- ▶ 向量处理器
- ▶ 深度学习处理器
 - ▶ 概述：研究意义与发展历史
 - ▶ 目标算法分析：计算特性与访存特性
 - ▶ 深度学习处理器结构
- ▶ 大规模深度学习处理器
- ▶ 总结

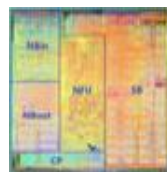
深度学习处理器的定位

能效

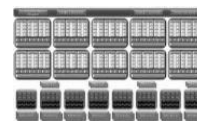
ASICs



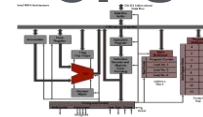
深度学习处理器



GPU



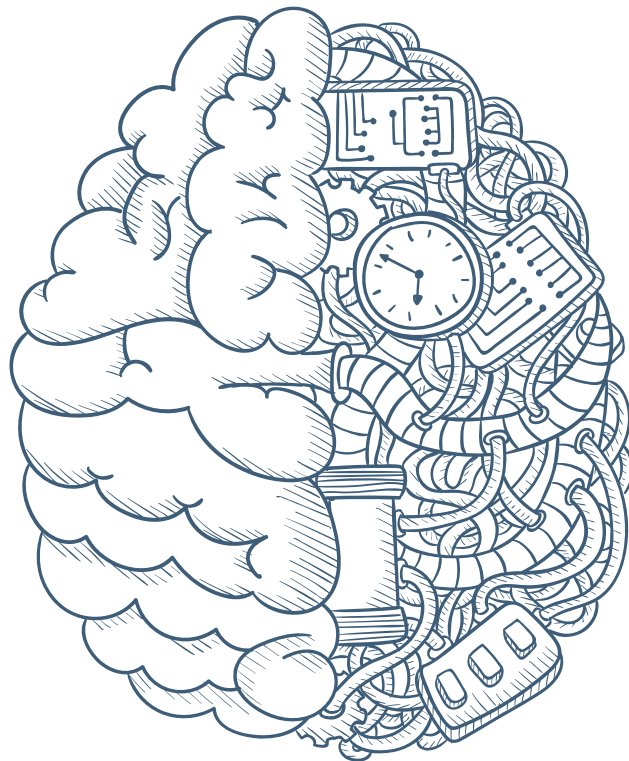
CPU



通用性

如何设计一个深度学习处理器DLP

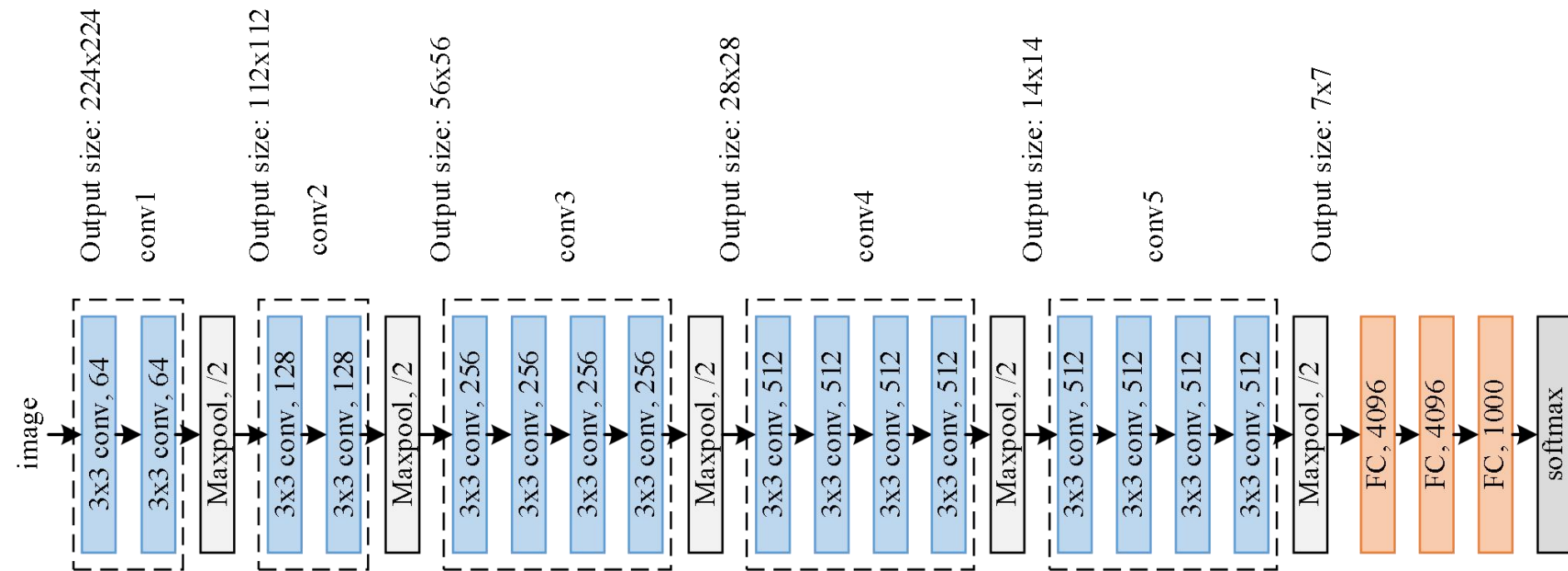
- ▶ 目标?
- ▶ 体系结构?
- ▶ 微体系结构?
- ▶ 可编程性?



设计思路

- ▶ 整个体系结构最重要的问题
 - ▶ 算法范围界定
 - ▶ 算法分析（计算特性、访存特性）
- ▶ 谁是我们的“朋友”？
 - ▶ 自定制硬件，可利用算法特性
 - ▶ 高效率
- ▶ 谁是我们的“敌人”？
 - ▶ 阻碍高效率：带宽，访存速度，访存代价

VGG19卷积神经网络

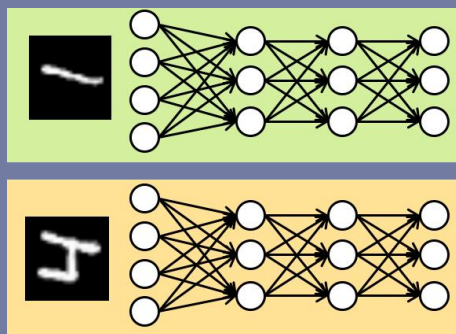


目标算法分析

- ▶ 如何兼顾高能效和广泛的适用性？
- ▶ 首先应该分析处理器服务于什么样的算法，目标算法具有哪些特性。

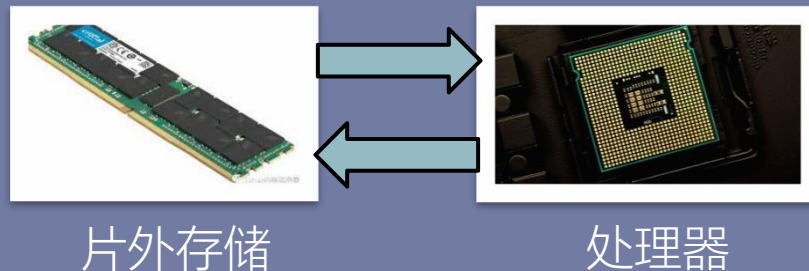
计算

- 是否存在固定重复的计算模式

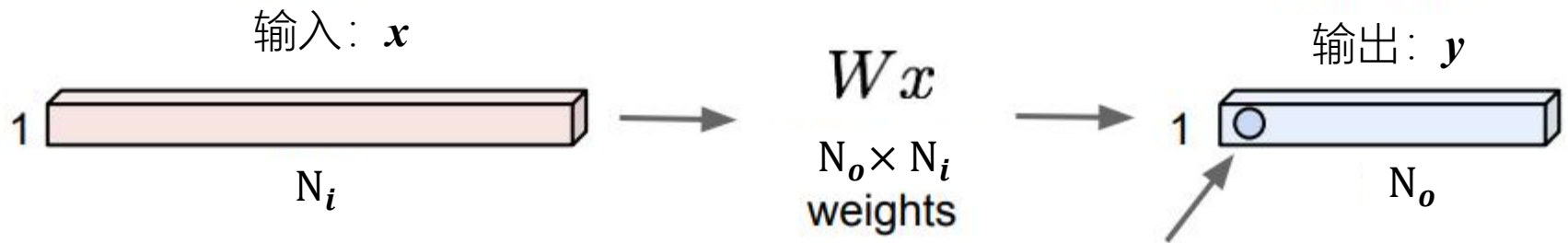


访存

- 对于带宽的需求、数据和计算的关系



全连接层



$$y[j] = G \left(b[j] + \sum_{i=0}^{N_i-1} W[j][i] \times x[i] \right)$$

*Source from Feifei Li CS231N (http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf)

全连接层

▶ 代码实现

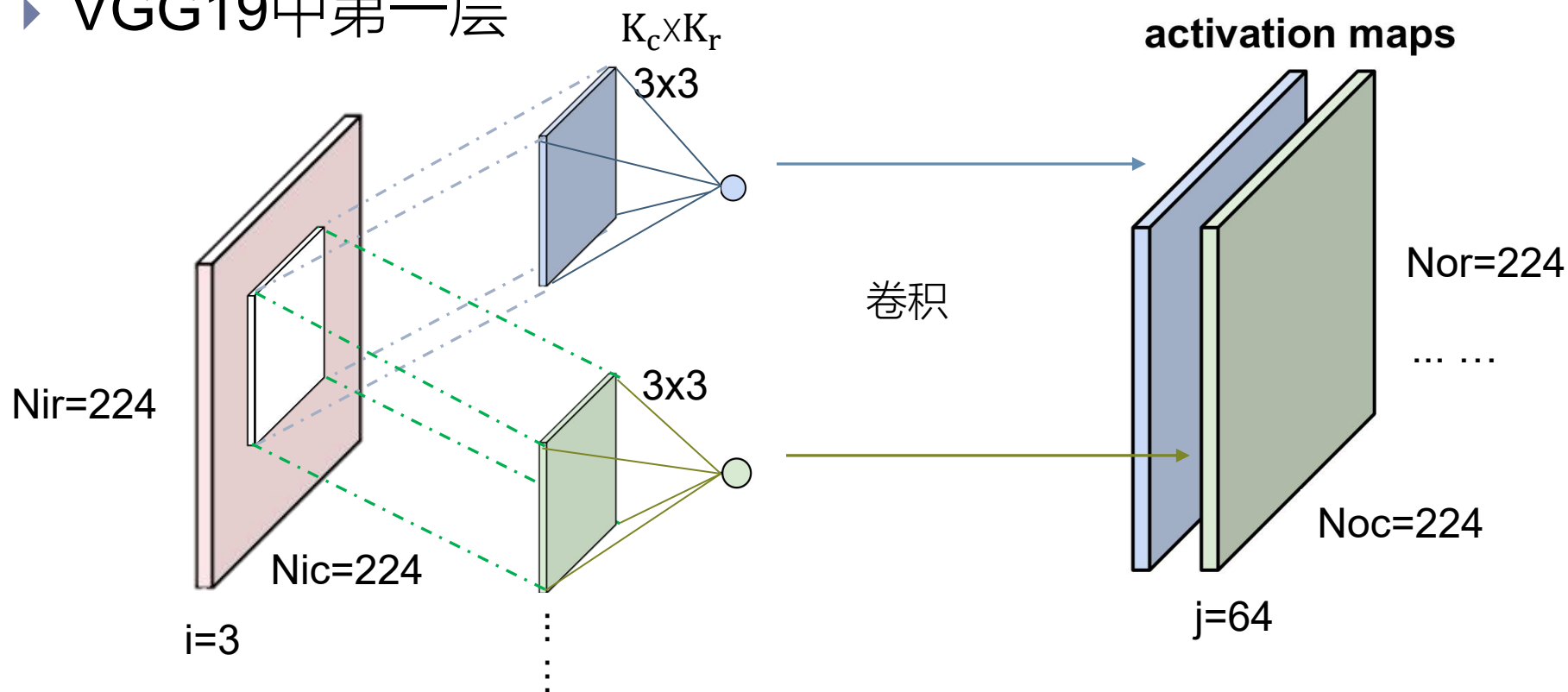
```
1 //x是输入神经元，y是输出神经元，W是权重
2 y(all) = 0; //// 初始化所有输出神经元
3 for (j=0; j<No; j++)
4     for (i=0; i<Ni; i++){
5         y[j]+=W[j][i]*x[i];
6         if (i==Ni)
7             y[j]=G(y[j]+b[j]);
8     }
```

▶ 计算特点

- ▶ 向量内积、向量的元素操作
- ▶ 无复杂控制流

卷积层

► VGG19中第一层



$$Y[nor][noc][j] = G \left(\mathbf{b}[j] + \sum_{i=0}^{N_{if}-1} \sum_{k_c=0}^{K_c-1} \sum_{k_r=0}^{K_r-1} \mathbf{W}[k_r][k_c][j][i] \times X[r+k_r][c+k_c][i] \right)$$

*Source from Feifei Li CS231N (http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf)

卷积层

```
1 nor = 0;
2 for (r=0; r<Nir; r+=sr) { // sr是垂直方向的卷积步长
3     noc = 0;
4     for (c=0; c<Nic; c+=sc) { // sc是水平方向的卷积步长
5         for (j=0; j<Nof; j++)
6             sum[j]=0;
7         for (kr=0; kr<Kr; kr++)
8             for (kc=0; kc<Kc; kc++)
9                 for (j=0; j<Nof; j++)
10                    for (i=0; i<Nif; i++)
11                        sum[j]+=W[kr][kc][j][i]*X[r+kr][c+kc][i];
12         for (j=0; j<Nof; j++)
13             Y[nor][noc][j]=G(sum[j]+b[j]);
14         noc++;}
15     nor++;
16 }
```

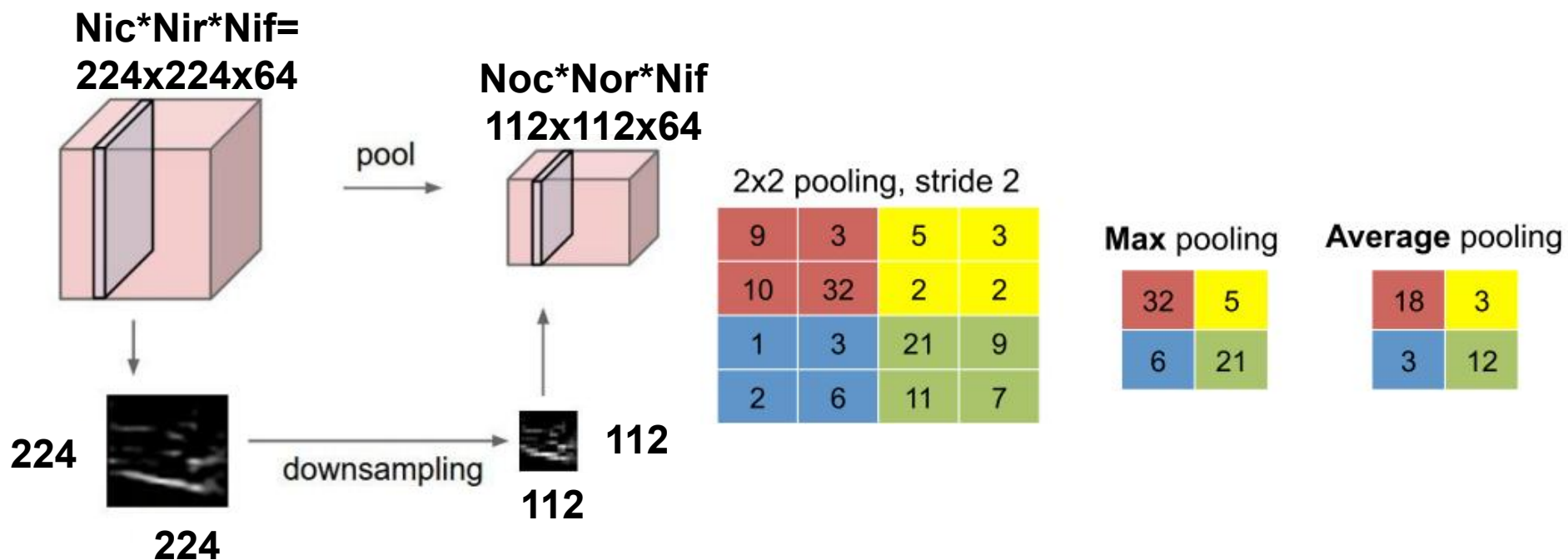
卷积层

```
1 nor = 0;
2 for (r=0; r<Nir; r+=sr) { // sr是垂直方向的卷积步长
3     noc = 0;
4     for (c=0; c<Nic; c+=sc) { // sc是水平方向的卷积步长
5         for (j=0; j<Nof; j++)
6             sum[j]=0;
7         for (kr=0; kr<Kr; kr++)
8             for (kc=0; kc<Kc; kc++)
9                 for (j=0; j<Nof; j++)
10                    for (i=0; i<Nif; i++)
11                        sum[j]+=W[kr][kc][j][i]*X[r+kr][c+kc][i];
12     }
13 }
```

► 计算特点

- 矩阵内积、向量的元素操作
- 无复杂控制流

池化层



MAX pooling:
$$Y[nor][noc][i] = \max_{0 \leq kc < K_c, 0 \leq kr < K_r} (X[r + kr][c + kc][i])$$

AVG pooling:
$$Y[nor][noc][i] = \frac{1}{K_c \times K_r} \sum_{k_c=0}^{K_c-1} \sum_{k_r=0}^{K_r-1} X[r + kr][c + kc][i]$$

*Source from Feifei Li CS231N (http://cs231n.stanford.edu/slides/2018/cs231n_2018_lecture05.pdf)

池化层

```
1  nor = 0;
2  for (r=0; r<Nir; r+=sr) { // sr是垂直方向的池化步长
3      noc = 0;
4      for (c=0; c<Nic; c+=sc) { // sc是水平方向的池化步长
5          for (i=0; i<Nif; i++)
6              value[i]=0;
7          for (kr=0; kr<Kr; kr++)
8              for (kc=0; kc<Kc; kc++)
9                  for (i=0; i<Nif; i++) {
10                     // for average pooling
11                     value[i]+=X[r+kr][c+kc][i];
12                     // for max pooling
13                     value[i] = max(value[i], X[r+kr][c+kc][i]);}
14
15          for (i=0; i<Nif; i++)
16              // for average pooling
17              Y[nor][noc][i]=value[i]/Kr/Kc;
18              // for max pooling
19              Y[nor][noc][i]=value[i];
20          noc++;}
21      nor++;
22 }
```


池化层

```
1  nor = 0;
2  for (r=0; r<Nir; r+=sr) { // sr是垂直方向的池化步长
3      noc = 0;
4      for (c=0; c<Nic; c+=sc) { // sc是水平方向的池化步长
5          for (i=0; i<Nif; i++)
6              value[i]=0;
7          for (kr=0; kr<Kr; kr++)
8              for (kc=0; kc<Kc; kc++)
9                  for (i=0; i<Nif; i++) {
10                     // for average pooling
11                     value[i]+=X[r+kr][c+kc][i];
12                     // for max pooling
13                     value[i] = max(value[i], X[r+kr][c+kc][i]);}
14      }
```

► 计算特点

- 向量的元素操作
- 无复杂控制流

卷积神经网络

► VGG19: 在风格迁移算法中使用

	VGG19
参数	1.14 (亿)
层类型	卷积, 池化, 全连接
计算过程	简洁
层数	25 (19+6)
卷积层	16 (3x3卷积核, 图大小不变)
池化层	5 (Max Pooling)
全连接层	3

E
19 weight layers
input (224 × 224 RGB image)
conv3-64
conv3-64
maxpool
conv3-128
conv3-128
maxpool
conv3-256
conv3-256
conv3-256
conv3-256
maxpool
conv3-512
conv3-512
conv3-512
conv3-512
maxpool
conv3-512
conv3-512
conv3-512
conv3-512
maxpool
FC-4096
FC-4096
FC-1000
soft-max

卷积神经网络

► 计算特征

不同层的计算特点

层	计算类型	乘加操作个数	激活函数操作个数
卷积层	矩阵内积，向量的元素操作	$N_{if} \times N_{of} \times N_{or} \times N_{oc} \times K_r \times K_c$ 个乘加	$N_{of} \times N_{or} \times N_{oc}$
池化层	向量的元素操作	$N_{if} \times N_{or} \times N_{oc} \times K_r \times K_c$ 个加法或比较 + $N_{if} \times N_{or} \times N_{oc}$ 个除法操作（平均池化）	无
全连接层	矩阵乘向量，向量的元素操作	$N_o \times N_i$ 个乘加	N_o



计算指令
硬件加速单元

卷积神经网络

- ▶ 访存特征
 - ▶ 如何降低访存带宽需求?
 - ▶ 提升数据访问的**局部性**
 - ▶ 典型方法: **循环分块**

访存特征分析-以全连接层为例

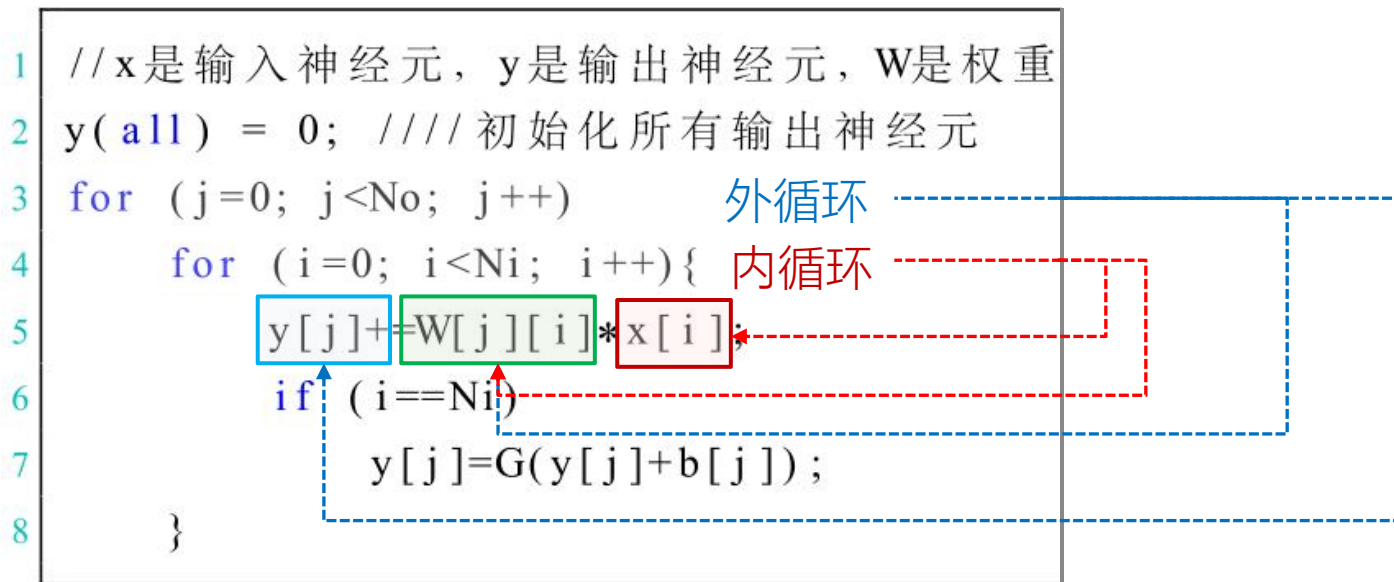
访存特征分析

全连接层代码实现：

```
1 //x是输入神经元，y是输出神经元，W是权重
2 y(all) = 0; //// 初始化所有输出神经元
3 for (j=0; j<No; j++)          外循环
4     for (i=0; i<Ni; i++){      内循环
5         y[j] += W[j][i] * x[i];
6         if (i==Ni)
7             y[j] = G(y[j] + b[j]);
8     }
```

访存特征分析-以全连接层为例

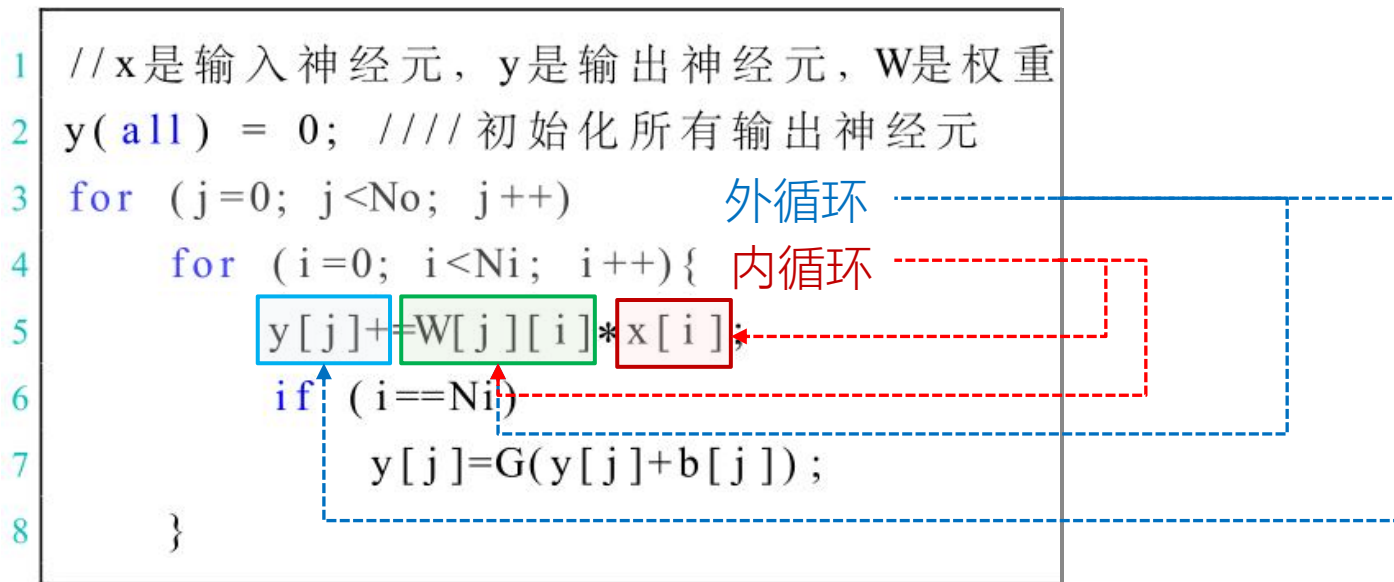
访存特征



外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$

访存特征分析-以全连接层为例

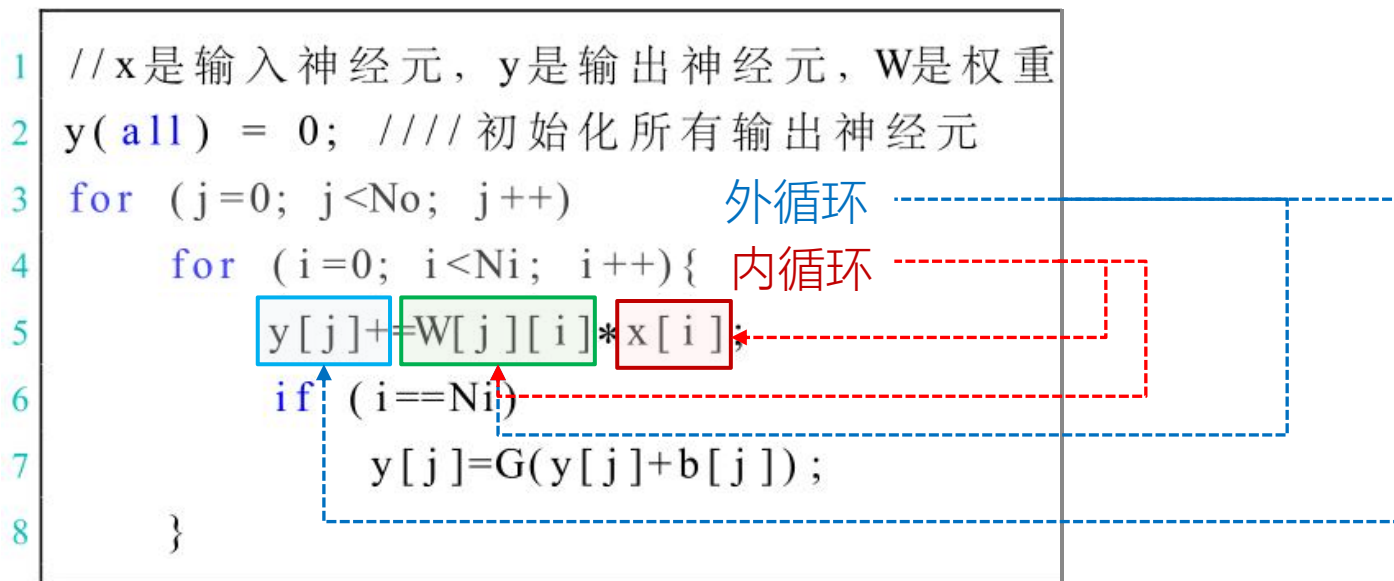
▶ 访存特征



外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$

访存特征分析-以全连接层为例

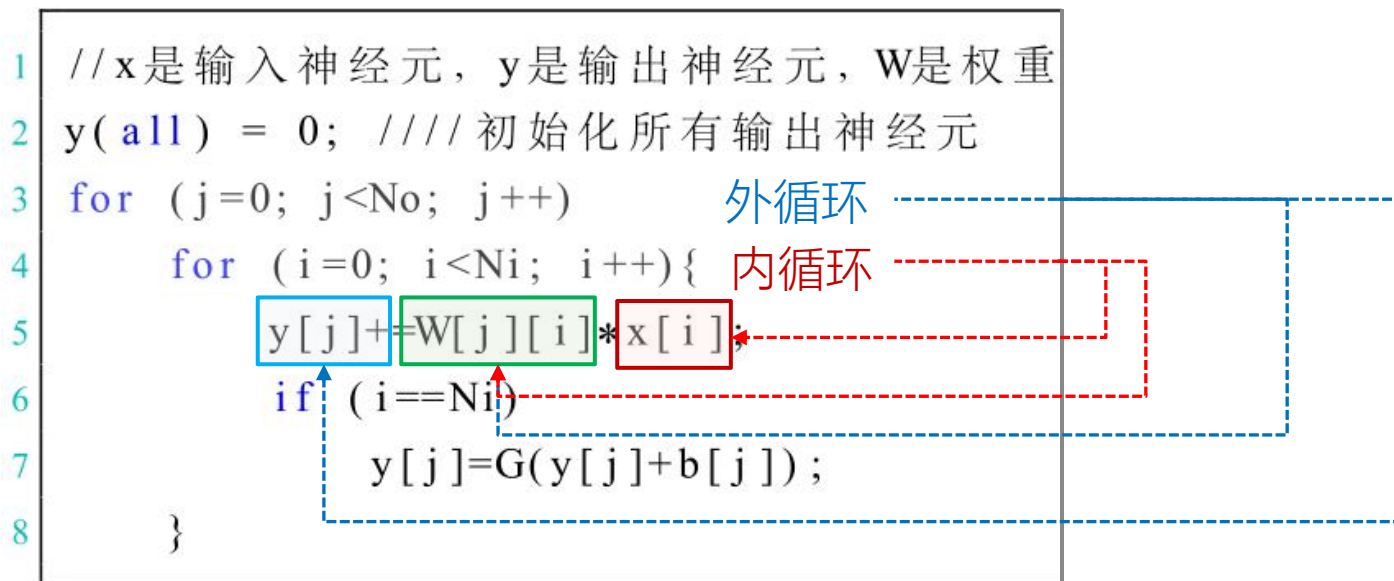
访存特征



外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$
	$i = N_i - 1$	$x[N_i - 1]$	$W[0][N_i - 1]$	$y[0]$	访存读 $x[N_i - 1]$, $W[0][N_i - 1]$, 存 $y[0]$

访存特征分析-以全连接层为例

访存特征



外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$
	$i = N_i - 1$	$x[N_i - 1]$	$W[0][N_i - 1]$	$y[0]$	访存读 $x[N_i - 1]$, $W[0][N_i - 1]$, 存 $y[0]$
$j = 1$	$i = 0$	$x[0]$	$W[1][0]$	$y[1]$	访存读 $x[0]$, $W[1][0]$, $y[1]$

访存特征分析-以全连接层为例

▶ 访存特征

- ▶ 分析 $x[i]$: 只有在计算下一个输出神经元的时候, 输入神经元 $x[i]$ 才会被读取进来

外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$
	...	...	...	...	...
	$i = N_i - 1$	$x[N_i - 1]$	$W[0][N_i - 1]$	$y[0]$	访存读 $x[N_i - 1]$, $W[0][N_i - 1]$, 存 $y[0]$
$j = 1$	$i = 0$	$x[0]$	$W[1][0]$	$y[1]$	访存读 $x[0]$, $W[1][0]$, $y[1]$

访存特征分析-以全连接层为例

访存特征

- 分析 $y[j]$: 在内层循环中, 每循环一次, $y[j]$ 都会被重用一次

外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$
	...	...	...	...	...
	$i = N_i - 1$	$x[N_i - 1]$	$W[0][N_i - 1]$	$y[0]$	访存读 $x[N_i - 1]$, $W[0][N_i - 1]$, 存 $y[0]$
$j = 1$	$i = 0$	$x[0]$	$W[1][0]$	$y[1]$	访存读 $x[0]$, $W[1][0]$, $y[1]$

访存特征分析-以全连接层为例

▶ 访存特征

- ▶ 分析 $W[j][i]$: 权重不会被重复使用

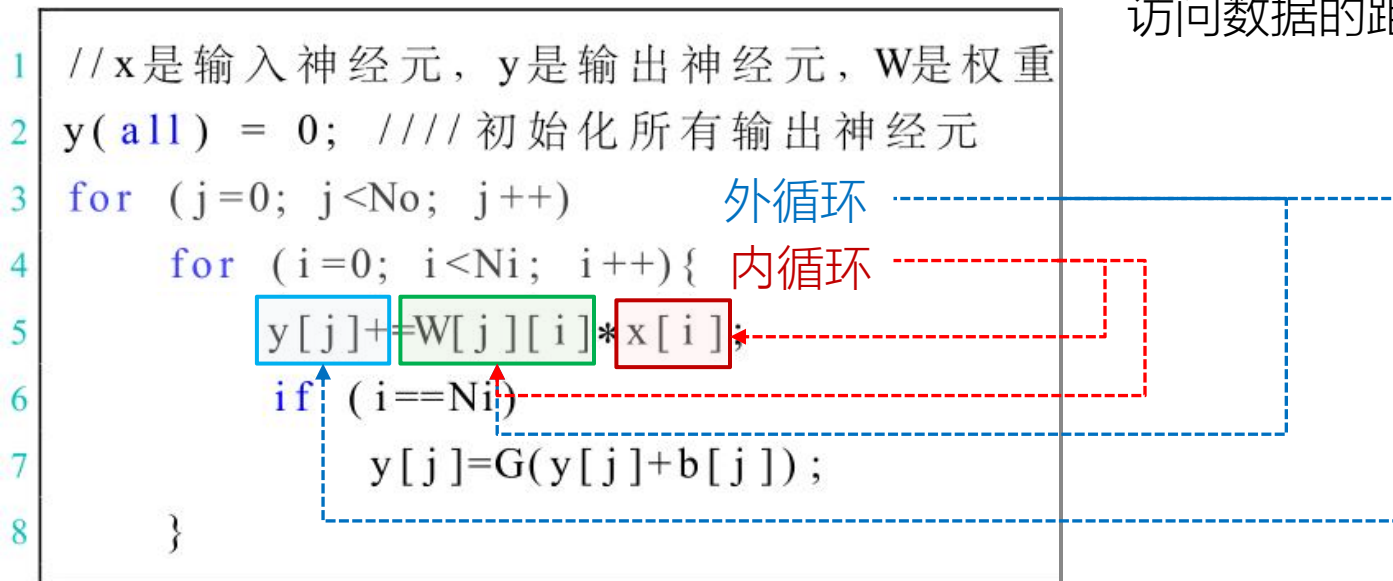
外循环	内循环	输入神经元	权重	输出神经元	访存行为
$j = 0$	$i = 0$	$x[0]$	$W[0][0]$	$y[0]$	访存读 $x[0]$, $W[0][0]$, $y[0]$
	$i = 1$	$x[1]$	$W[0][1]$	$y[0]$	访存读 $x[1]$, $W[0][1]$
	...	...	...	...	...
	$i = N_i - 1$	$x[N_i - 1]$	$W[0][N_i - 1]$	$y[0]$	访存读 $x[N_i - 1]$, $W[0][N_i - 1]$, 存 $y[0]$
$j = 1$	$i = 0$	$x[0]$	$W[1][0]$	$y[1]$	访存读 $x[0]$, $W[1][0]$, $y[1]$

卷积神经网络

访存特征

全连接层:

复用距离: 连续两次访问数据的距离



$x[i]$ 外循环复用, 复用距离等于 N_i

$W[j][i]$ 内外循环无复用

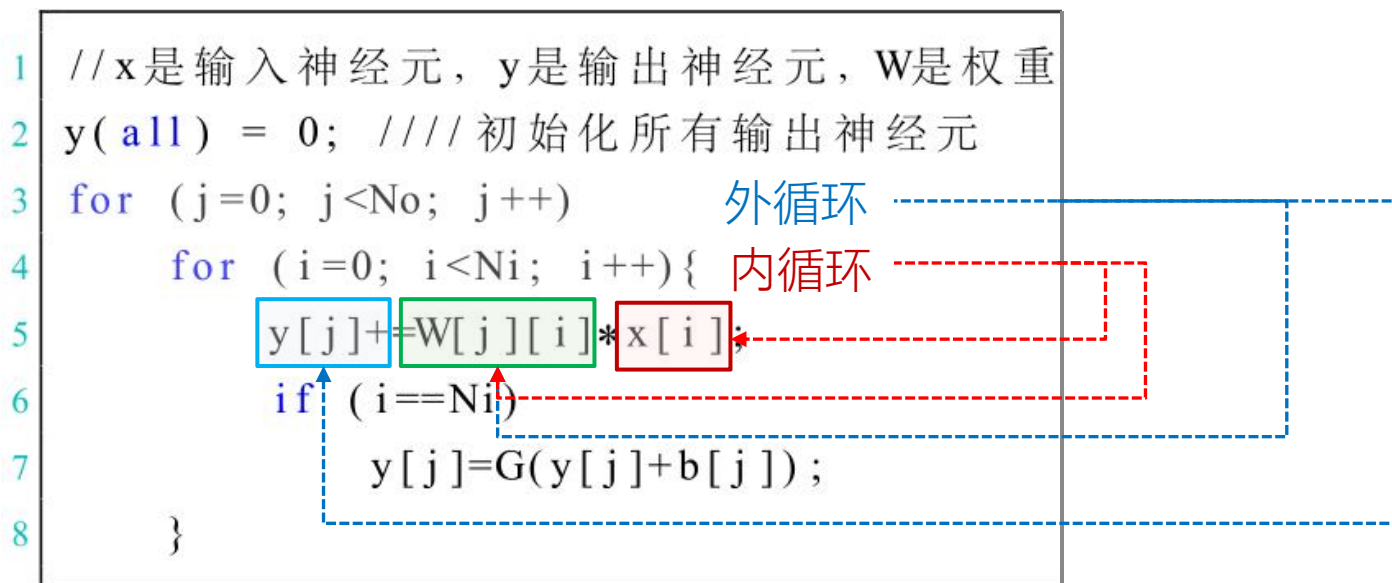
$y[j] +$ 内循环复用, 复用距离等于1



可解耦性
可复用性

卷积神经网络

访存特征



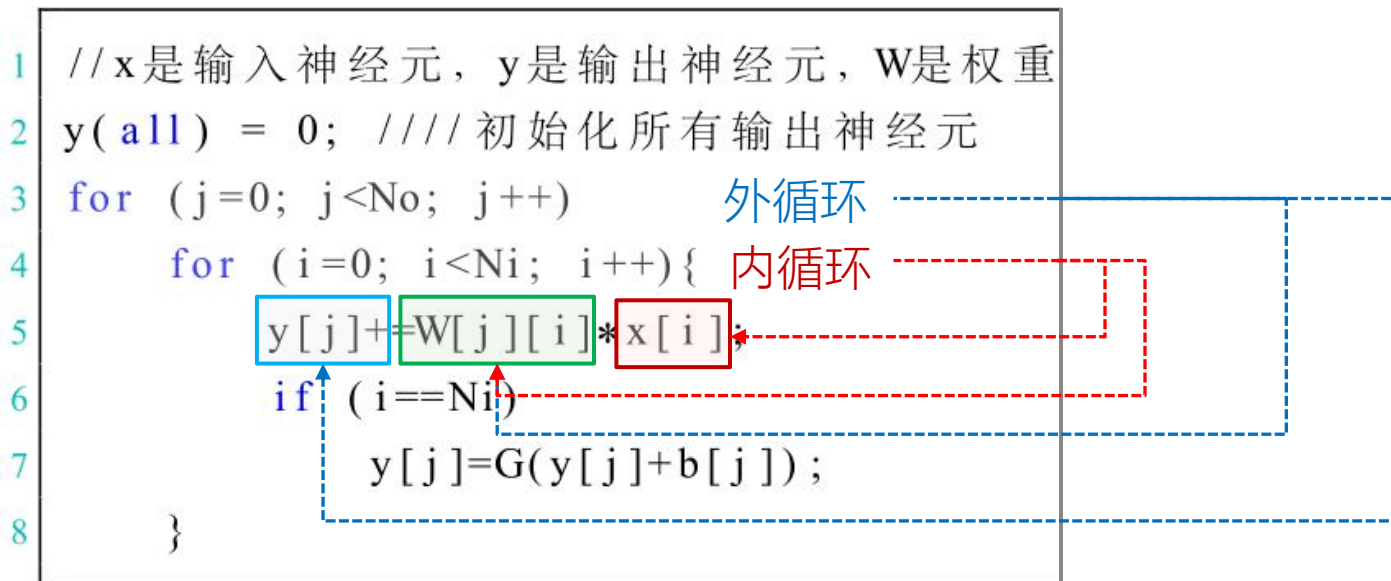
可解耦性：神经网络计算中权重、输入和输出神经元访存行为不同，可以单独拆分出不同的访存数据流

可复用性：网络中的权重、输入输出神经元（中间结果）在计算过程中都可以被多次使用

卷积神经网络

▶ 访存特征

- ▶ 没有重用性或重用距离很大的数据所需要的访存带宽很高



x[i]

外循环复用，复用距离等于Ni

W[j][i]

内外循环无复用

y[j] +

内循环复用，复用距离等于1

卷积神经网络

► 访存特征

VGG19最后两层

maxpool
FC-4096
FC-4096
FC-1000
soft-max

权值参数数量：

FC-4096: $4096 \times 4096 = 16777216$

FC-1000: $4096 \times 1000 = 4096000$

参数大小（浮点数）：

FC-4096: $16777216 \times 4 \text{ Byte} = 64\text{MB}$

FC-1000: $4096000 \times 4 \text{ Byte} = 15.625\text{MB}$



Intel Xeon 6130的L3 cache大小：

22MB

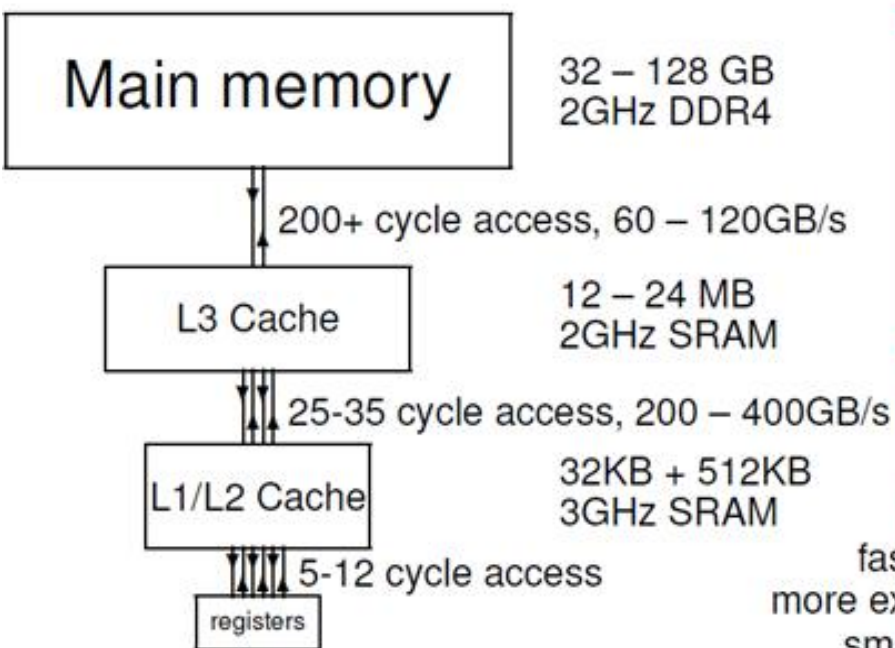
Nvidia V100片上L2 cache：

6MB

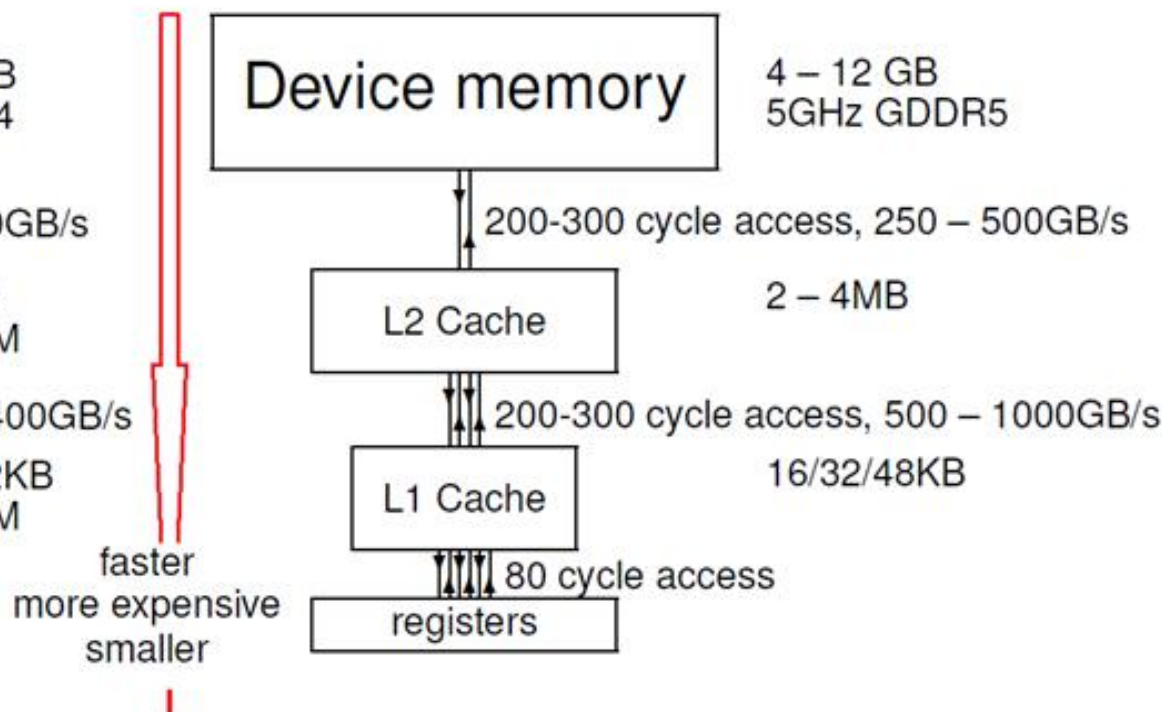
卷积神经网络

访存特征

CPU Memory Hierarchy



GPU Memory Hierarchy



卷积神经网络

► 访存特征

VGG19最后两层

maxpool
FC-4096
FC-4096
FC-1000
soft-max

权值参数数量：

FC-4096: $4096 \times 4096 = 16777216$

FC-1000: $4096 \times 1000 = 4096000$

参数大小（浮点数）：

FC-4096: $16777216 \times 4 \text{ Byte} = 64\text{MB}$

FC-1000: $4096000 \times 4 \text{ Byte} = 15.625\text{MB}$



Intel Xeon 6130的L3 cache大小: 22MB

Nvidia V100片上L2 cache: 6MB

权值数据量大无复用，带宽需求高，需要进行循环分块

卷积神经网络

► 访存特征

循环分块 (tiling)

```
1 //T是循环分块大小
2 y(all) = 0; //初始化所有输出神经元
3 for (ii=0; ii<Ni; ii+=Ti)
4     for (j=0; j<No; j++)
5         for (i=ii; i<ii+Ti; i++){
6             y[j]+=W[j][i]*x[i];
7             if (i==Ni)
8                 y[j]=G(y[j]+b[j]);
9         }
```

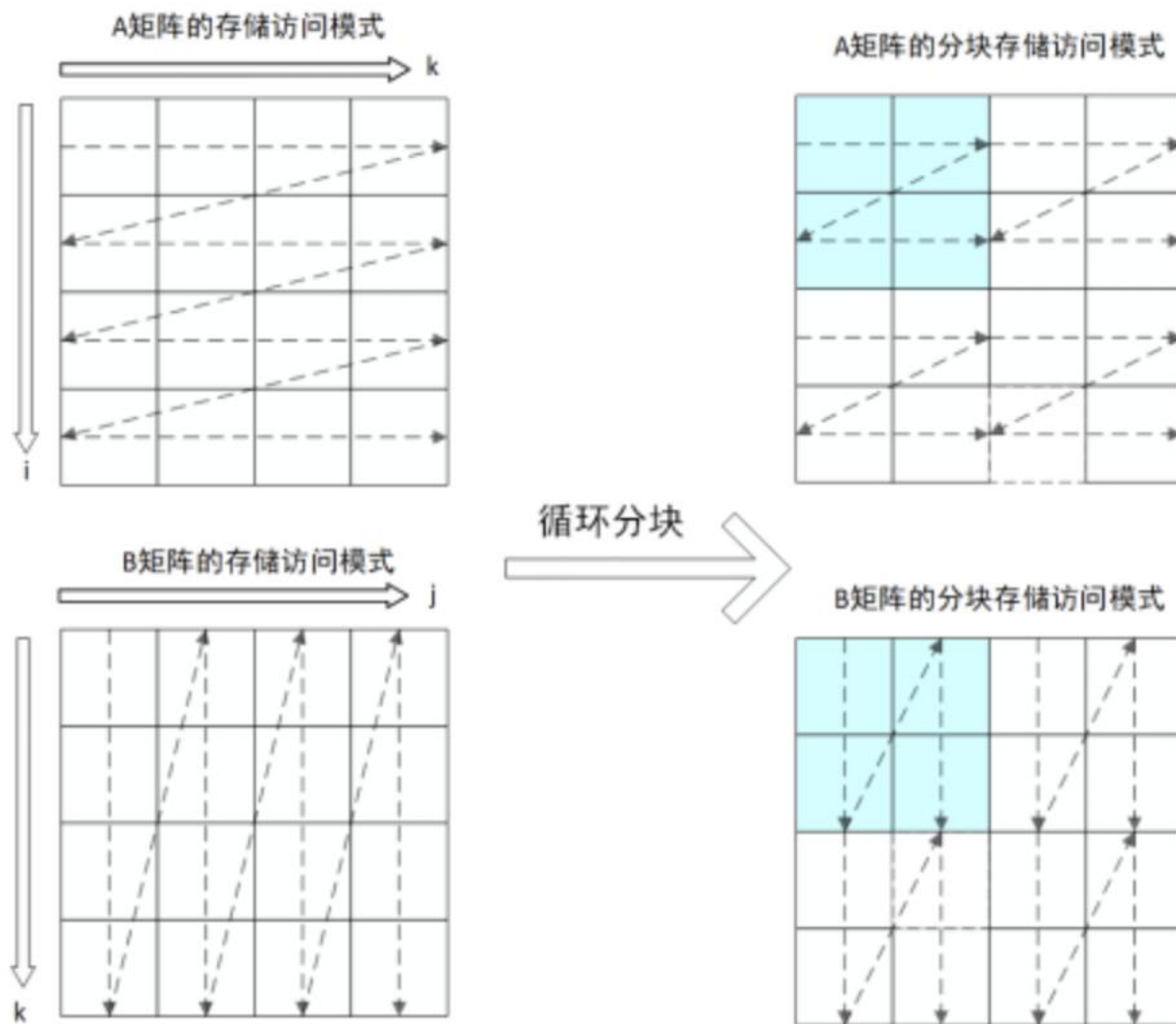
$y[j]$ 依旧是重用距离=1

循环分块前: $x[i]$ 每隔 N_i 次才会被重用一次, 如果片上缓存放不下 N_i 个神经元, 那么输入神经元一共需要 $N_o \times (N_i + 1)$ 次访存。

循环分块后: T_i 大小的数据可以放进缓存用于所有输输出神经元的计算, 因此输入神经元只需要 $N_i + 1$ 次访存, $x[i]$ 重用距离降低为 T_i 。

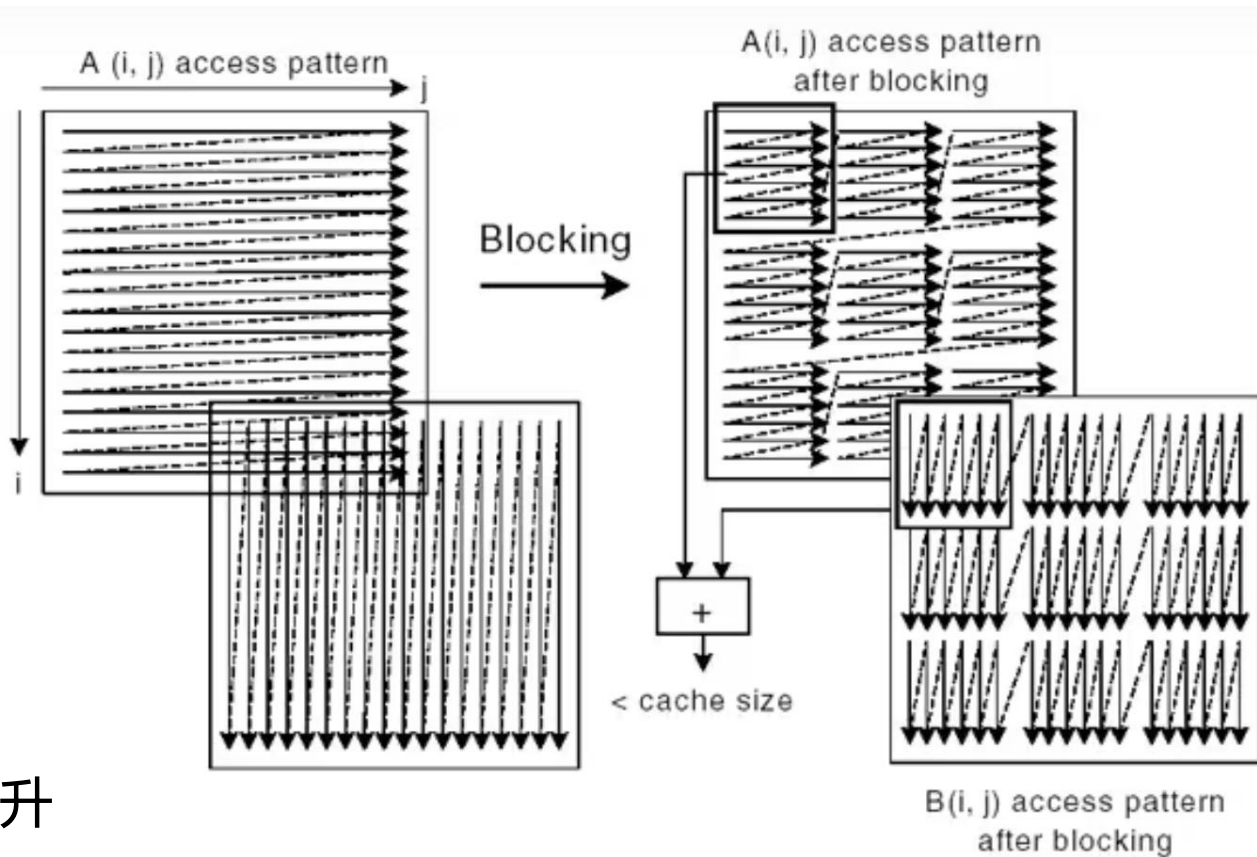
卷积神经网络

► 访存特征



卷积神经网络

访存特征



Cache命中率提升

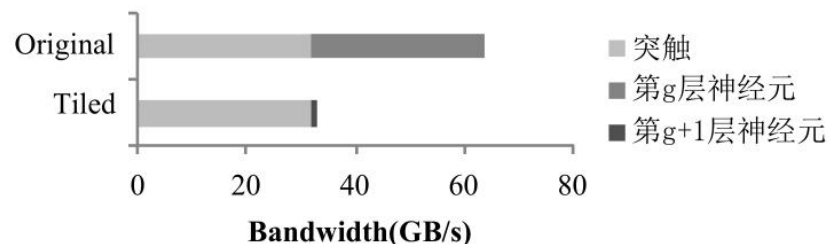
处理器在访问内存时首先判断所要访问的内容是否在**Cache**中，如果在，就称为“命中”，此时处理器直接从**Cache**中调用该内容；否则，就称为“不命中”，处理器只好去内存中调用所需的子程序或指令了。

卷积神经网络

► 访存特征

循环分块 (tiling)

```
1 //T是循环分块大小
2 y(all) = 0; // 初始化所有输出神经元
3 for (ii=0; ii<Ni; ii+=Ti)
4     for (j=0; j<No; j++)
5         for (i=ii; i<ii+Ti; i++){
6             y[j]+=W[j][i]*x[i];
7             if (i==Ni)
8                 y[j]=G(y[j]+b[j]);
9         }
```



假设 $N_i=16384$

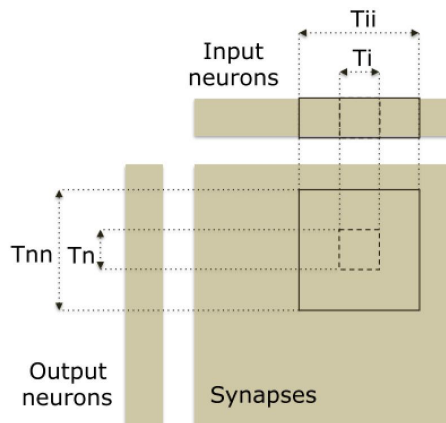
循环分块能减少46.7%的访存带宽需求

卷积神经网络

访存特征

全连接层做两层循环分块 (tiling)

```
1 for (jjj=0; jjj <No; jjj+=Tjj) { // 对输出神经元进行分块, Tjj和Tj是两层分块大小
2   for (jj=jjj; jj < jjj+Tjj; jj+=Tj) {
3     for (j=jj; j < jj+Tj; j++)
4       y[j]=0;
5     for (iii=0; iii < Ni; iii+=Tii) { // 对输入神经元进行分块, Tii和Ti是两层分块大小
6       for (ii=iii; ii < iii+Tii; ii+=Ti)
7         for (j=jj; j < jj+Tj; j++)
8           for (i=ii; i < ii+Ti; i++)
9             sum[j] += W[j][i]*x[i];
10    for (j=jj; j < jj+Tj; j++)
11      y[j]=G(sum[j]+b[j]);
12  }}
```



现有编译器
可以自动处
理一些循环
分块操作

卷积神经网络

► 访存特征

池化层做循环分块 (tiling)

```
1  for (rr=0; rr<Nir; rr+=Tr) { //对输入特征图的垂直方向进行分块
2      for (cc=0; cc<Nic; cc+=Tc){ //对输入特征图的水平方向进行分块
3          for (iii=0; iii<Ni; iii+=Tii){ //对输入特征图通道进行分块，Tii为外层循环分块
              大小
4              nor = 0;
5              for (r=rr; r<rr+Tr; r+=sr){
6                  noc = 0;
7                  for (c=cc; c<cc+Tc; c+=sc){
8                      for (ii=iii; ii<iii+Tii; ii+=Ti){ //对输入特征图的通道进一步分块，Ti
                          为内层循环分块大小
9                          for (i=ii; i<ii+Ti; i++)
10                             value[i]=0;
11                          for (kr=0; kr<Kr; kr++)
12                             for (kc=0; kc<Kc; kc++)
13                                 for (i=ii; i<ii+Ti; i++) {
14                                     // for average pooling
15                                     value[i]+=X[r+kr][c+kc][i];
16                                     // for max pooling
17                                     value[i] = max(value[i], X[r+kr][c+kc][i]);}
18                          for (i=ii; i<ii+Ti; i++)
19                              // for average pooling
20                              Y[noc][nor][i]=value[i]/Kr/Kc;
21                              // for max pooling
22                              Y[noc][nor][i]=value[i];}
23                      noc++;}
24                  nor++;
25          }}}}

```


卷积神经网络

► 访存特征

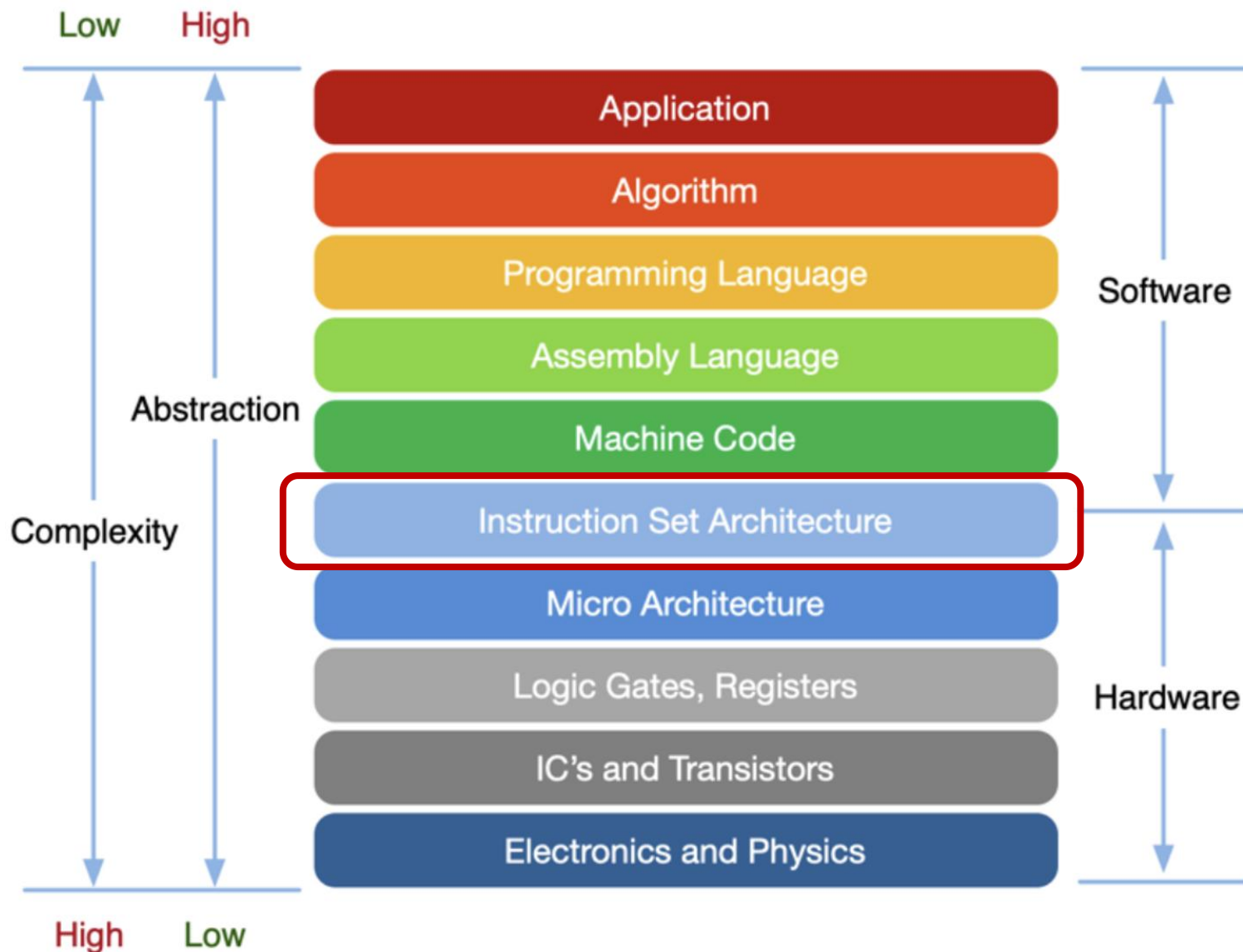
不同层的重用特性

层	可重用	不可重用
卷积层	输入神经元、输出神经元、突触权重	无
池化层	当池化窗口大于步长，部分输入神经元可重用	池化窗口小于等于步长时，输入神经元、输出神经元都不可重用
全连接层	输入神经元、输出神经元	突触权重

提纲

- ▶ 通用处理器
- ▶ 向量处理器
- ▶ 深度学习处理器
 - ▶ 概述：研究意义与发展历史
 - ▶ 目标算法分析：计算特性与访存特性
 - ▶ 深度学习处理器指令集
- ▶ 大规模深度学习处理器
- ▶ 总结

体系结构层次



硬连线 v.s. 指令集

- ▶ 如何让一个芯片支持深度学习算法的不同算子？
- ▶ 硬连线：为每个神经网络层/算子提供专门的硬件控制逻辑
 - ▶ 泛化性差，无法支持新的算子
- ▶ 指令集：把算子拆分成一些基本操作，每个操作对应一条指令
 - ▶ 可以轻松地支持新的算子

指令集

- ▶ 软硬件的唯一接口
- ▶ 对于程序员而言：并不需要关心硬件具体设计成什么样，只需要知道面向哪一套指令集编程即可。（例如针对X86的CPU写的程序，可以同时运行在286、386、486到现在的i7、i5上面运行。）
- ▶ 对于硬件而言：指令集是最重要的优化加速目标，有了指令集，硬件设计就有了方向。

指令集

- ▶ 计算机的抽象模型
- ▶ 定义了体系结构
- ▶ 软硬件的唯一接口
- ▶ 为什么采用指令集
 - ▶ 灵活性：支持未来可能出现的新的深度学习算法
 - ▶ 通用性：支持广泛的深度学习算法

指令集

- ▶ 通用CPU指令集：分为RISC(Reduced Instruction Set Computer, 精简指令集计算机)、CISC(Complex Instruction Set Computer, 复杂指令集计算机)
- ▶ RISC: MIPS、ARM、RISC-V; 一条指令只完成简单任务

▶	CISC	RISC
出现时间	20世纪60年代	20世纪80年代
指令功能	复杂	简单
指令长度	多种指令长度	固定指令长度
指令数	多	较少
寻址模式	支持多种寻址模式	寄存器基址加偏移量

指令集

► 设计原则

- 深度学习算法特点：可向量化操作 -> 并行化

不同层的计算特点

层	计算类型	乘加操作个数	激活函数操作个数
卷积层	矩阵内积，向量的元素操作	$N_{if} \times N_{of} \times N_{or} \times N_{oc} \times K_r \times K_c$ 个乘加	$N_{of} \times N_{or} \times N_{oc}$
池化层	向量的元素操作	$N_{if} \times N_{or} \times N_{oc} \times K_r \times K_c$ 个加法或比较 + $N_{if} \times N_{or} \times N_{oc}$ 个除法操作（平均池化）	无
全连接层	矩阵乘向量，向量的元素操作	$N_o \times N_i$ 个乘加	N_o

指令集

- ▶ 常见思路：
 - ▶ 数据级并行：一条指令同时处理多个数据
 - ▶ 指令数较少
 - ▶ 指令流水线功耗、面积开销小
 - ▶ 灵活性不足
 - ▶ 指令级并行：每个指令只输出一个数，运行时动态调用
 - ▶ 灵活性高
 - ▶ 指令流水线控制通路复杂
 - ▶ 功耗、面积开销大
- ▶ ---> 数据级并行

指令集

► DLP指令集

指令类型		例子	操作对象
控制指令		跳转(JUMP), 条件分支(CB)	寄存器(标量值), 立即数
数据转移指令	矩阵(Matrix)	矩阵 取(MLOAD)/存(MTORE)/移动(MMOVE)	寄存器(矩阵 地址/大小, 标量值), 立即数
	向量(Vector)	向量 取(VLOAD)/存(VSTORE)/移动(VMOVE)	寄存器(向量 地址/大小, 标量值), 立即数
	标量(Scalar)	标量 取(SLOAD)/存(SSTORE)/移动(SMOVE)	寄存器(标量值), 立即数
计算指令	矩阵(Matrix)	矩阵乘向量(MMV), 向量乘矩阵(VMM), 矩阵乘标量(MMS), 外积(OP), 矩阵相加(MAM), 矩阵相减(MSM)	寄存器(矩阵/向量 地址/大小, 标量值)
	向量(Vector)	向量基本运算(加(VAV)、减(VSV)、乘(VMV)、除(VDV)), 向量超越函数(指数(VEXP)、对数(VLOG)), 内积(IP), 随机向量(RV), 向量最大值(VMAX), 向量最小值(VMIN)	寄存器(向量 地址/大小, 标量值)
	标量(Scalar)	标量基本运算, 标量超越函数	寄存器(向量 地址/大小, 标量)
逻辑运算指令	向量(Vector)	向量比较(大于(VGT),等于(VE)),向量逻辑操作(与(VAND), 或(VOR), 取反(VNOT)), 向量最值归约(VGTM)	寄存器(向量 地址/大小, 标量)
	标量(Scalar)	标量比较, 标量逻辑运算	寄存器(标量), 立即数

Load-store结构: 只通过load和store指令访问主存

64-bit定长指令, 变长操作数 (可以完成变长功能, 由寄存器指定长度)

指令集

▶ 基本概念

▶ 操作数(Operands): 三种基本类型

- ▶ 寄存器操作数: 用已命名的寄存器
- ▶ 存储操作数: 引用内存位置
- ▶ 立即操作数: 常量 (Common case faster的思想)

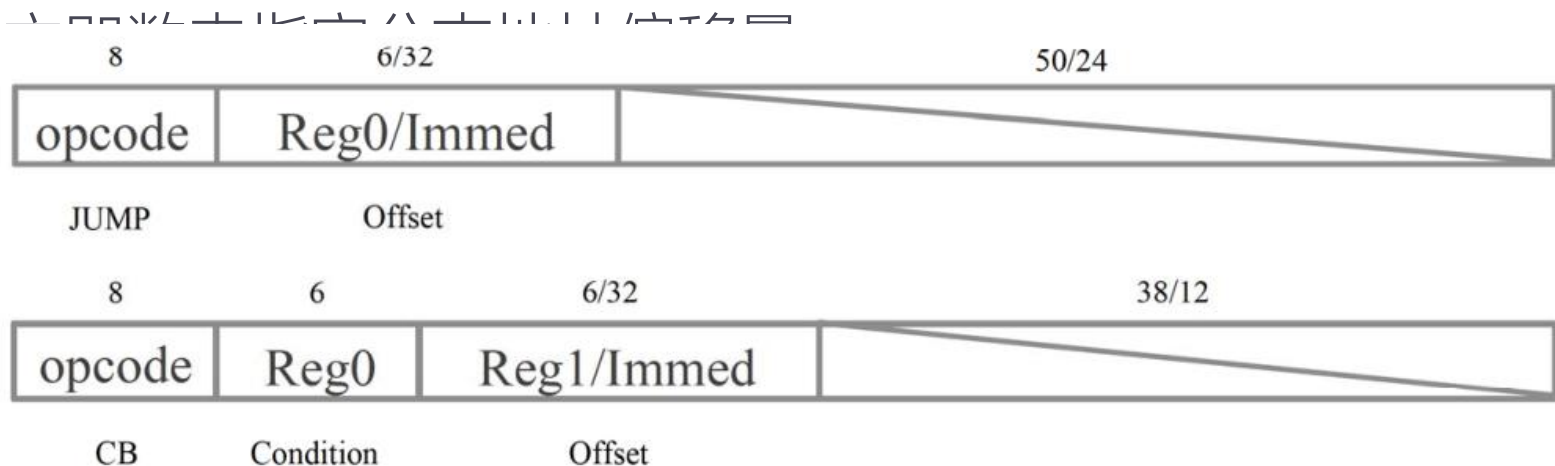
▶ Load-store结构: 只有load-store两个操作需要访存, 其他所有的操作数都来自寄存器而无须访存, 所以只需要对load-store优化, 也是RISC的特点之一

▶ 定长指令: RISC一般是定长指令, x86包含变长指令, 变长的指令会导致译码变得复杂

指令集

▶ 控制指令

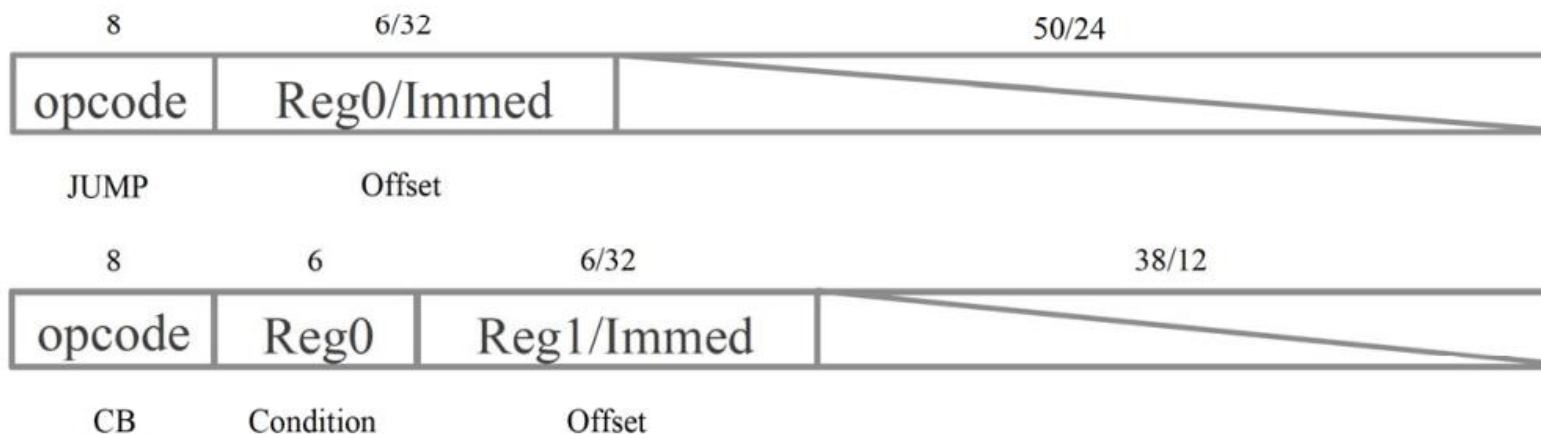
- ▶ **JUMP**: 立即跳转指令, 通过立即数或通用寄存器来指定偏移量
- ▶ **CB**: 条件分支指令, 用通用寄存器存放条件, 用通用寄存器或



指令集

▶ 控制指令

- ▶ **JUMP**: 跳到某个函数入口
- ▶ **CB**: 循环中判断是否继续执行还是跳出循环



指令集

▶ 控制指令

- ▶ 除了常规跳转，考虑到深度学习常常需要执行重复固定次数的循环，因此可以增加计数循环跳转指令
- ▶ **REPEAT**：计数循环跳转指令



指令集

▶ 数据传输指令

▶ Load/Store指令：主存和片上存储交互

▶ MLOAD/MSTORE：矩阵数据（变长）

▶ VLOAD/VSTORE：向量数据（变长）

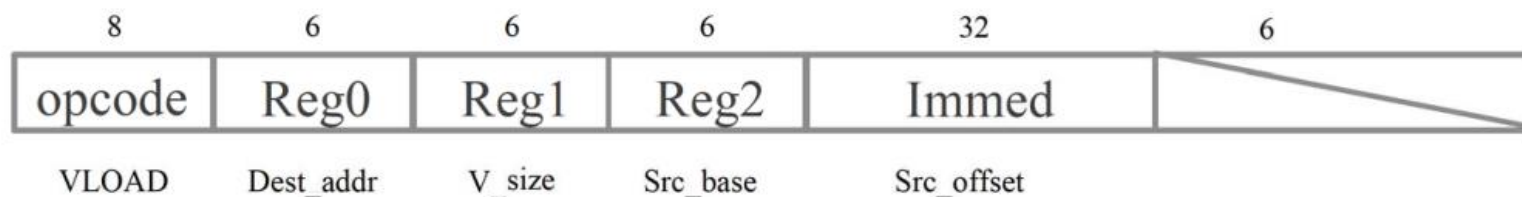
▶ SLOAD/SSTORE：标量数据

一条指令实现大量矩阵搬运，
那取指译码的开销就被摊薄了

▶ MOVE指令：片上数据传输

▶ MMOVE, VMOVE, SMOVE

将内存中的向量装到片上存储中
源数据地址是基地址+偏移量



命名方式：

MLOAD

Matrix-Load

VLOAD

Vector-Load

指令集

▶ 计算指令

命名方式: $y = Wx$

MMV

Matrix-**Multiply**-Vector

▶ 矩阵运算:

▶ MMV, VMM, MMS, OP (外积), MAM, MSM

▶ 向量运算:

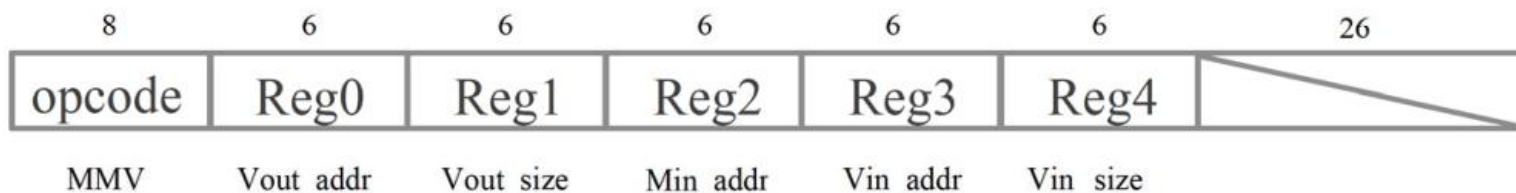
▶ VAV, VSV, VMV, VDV, VEXP (向量指数), VLOG (向量对数),
IP (内积), RV (随机向量生成), VMAX/VMIN (向量最值)

sigmoid

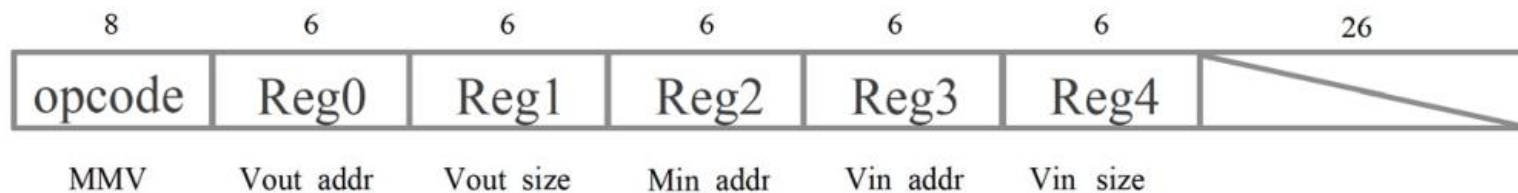
▶ 标量运算:

Dropout、随即采样

▶ 加减乘除基本运算, 标量超越函数



指令集



- ▶ Reg0: 指定输出向量在片上的基地址
- ▶ Reg1: 输出向量的大小
- ▶ Reg2: 指定输入矩阵的基地址
- ▶ Reg3: 指定输入向量的基地址
- ▶ Reg4: 指定输入向量的大小

指令集

▶ 逻辑指令

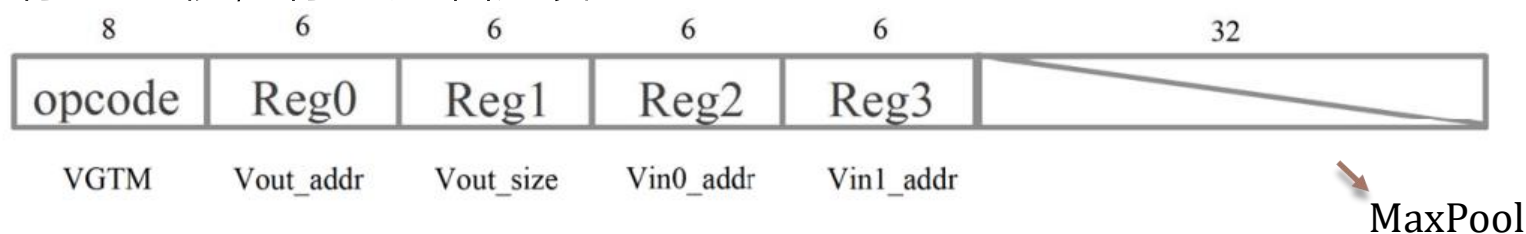
▶ 向量逻辑:

- ▶ 比较 (VGT, VE), 逻辑 (VAND, VOR, VNOT), 最值归约 VGTM

▶ 标量逻辑:

VGTM: Vector-Greater-Than-Merge

- ▶ 标量比较, 标量逻辑运算



最值归约: $Vout[i] = (Vin0[i] > Vin1[i]) ? Vin0[i] : Vin1[i]$

DLP指令集代码示例

► 全连接层

```
1 Fully connection code:
2 // $0: 输入特征向量大小, $1: 输出特征向量大小, $2: 权重矩阵大小
3 // $3: 输入特征向量地址, $4: 权重地址
4 // $5: 偏置地址, $6: 输出特征向量地址
5 // $7-$10: 临时变量地址
6 VLOAD    $3, $0, #100 // 从地址 (100) 读取向量数据到片上存储地址 $3
7 MLOAD    $4, $2, #300 // 从地址 (300) 读取权重矩阵到片上存储地址 $4
8 MMV      $7, $1, $4, $3, $0 //  $Wx$ 
9 VAV      $8, $1, $7, $5      //  $t = Wx + b$ 
10 VEXP     $9, $1, $8          //  $e^t$ 
11 VAS      $10, $1, $9, #1     //  $1 + e^t$ 
12 VDV      $6, $1, $9, $10     //  $y = e^t / (1 + e^t)$ 
13 VSTORE   $6, $1, #200       // 输出向量存入地址 (200)
```

DLP代码示例

► 池化层

```
15 Pooling code:
16 // $0: 特征通道数  $N_i$ , $1: 输入特征图大小  $N_{ic} \times N_{ir} \times N_i$ 
17 // $2: 输出数据大小, $3: 池化窗口大小-1
18 // $4: 水平方向循环次数, $5: 垂直方向循环次数
19 // $6: 输入地址, $7: 输出地址
20 // $8: 输入特征图垂直方向步长
21     VLOAD    $6, $1, #100      // 从地址 (100) 读取输入特征图到片上存储地址 $6
22     SMOVE    $5, $3           // 初始化  $\$5 = K_r - 1$ 
23 L0: SMOVE    $4, $3           // 初始化  $\$4 = K_c - 1$ 
24 L1: VGTM     $7, $0, $6, $7
25 // feature map m, output[m]=(input[c][r][m]>output[m])?
26 // input[c][r][m]:output[m]
27     SADD     $6, $6, $0      // 更新输入向量地址
28     SADD     $4, $4, #-1     // c-
29     CB       #L1, $4        // if(c>0) goto L1
30     SADD     $6, $6, $8      // 更新输入向量地址
31     SADD     $5, $5, #-1     // r-
32     CB       #L0, $5        // if(r>0) goto L0
33     VSTORE   $7, $2, #200    // 输出向量存入地址 (200)
```

两层循环

每次VGTM比较两个 N_i 维度向量的对应元素



谢谢大家！